

CHƯƠNG 2: HỆ THỐNG FILE

1. Các khái niệm
2. Các phương pháp truy cập file
3. Các thao tác với file
4. Thư mục
5. Cấp phát không gian cho file
6. Quản lý không gian trống trên đĩa
7. Độ tin cậy của hệ thống file
8. Bảo mật cho hệ thống file
9. Hệ thống file FAT

- *File được định nghĩa như tập hợp các thông tin liên quan đến nhau được đặt tên và được lưu trữ trên bộ nhớ ngoài*
- Thuộc tính của file: để quản lý file ngoài nội dung, HĐH còn định nghĩa các thuộc tính, tính chất. File có các thuộc tính như sau:
 - Tên file
 - Kiểu file
 - Kích thước file
 - Người tạo file, người sở hữu
 - Quyền truy cập file
 - Thời gian tạo file, sửa file, truy cập lần cuối
 - Vị trí file

- Đặt tên cho file:
 - Cho phép xác định file
 - Là thông tin người dùng thường sử dụng nhất khi làm việc với file
 - Quy tắc đặt tên cho file của một số HDH:

Hệ điều hành	Độ dài tối đa	Phân biệt chữ hoa, chữ thường	Cho phép sử dụng dấu cách	Các ký tự cấm
MS-DOS	8 cho tên file 3 cho mở rộng	không	không	Bắt đầu bằng chữ cái hoặc số Không được chứa các ký tự / \ [] : ; = , ^ ? @
Windows NT FAT	255 ký tự cho cả tên file và đường dẫn	không	có	Bắt đầu bằng chữ cái hoặc số Không được chứa các ký tự / \ [] : ; = , ^ ? @
Windows NT NTFS	255	không	có	Không được chứa các ký tự / \ < > * :
Linux (EXT3)	256	Có	có (nếu tên file chứa trong ngoặc kép)	Không được chứa các ký tự ! @ # \$ % ^ & * () [] { } ' " / \ : ; < > `

- Các thông tin trong file có thể rất khác nhau
- Có những file chứa nhiều thông tin không có cấu trúc: file văn bản. File có cấu trúc như: file CSDL, file excel.
- => Cấu trúc của file cũng rất khác nhau và phụ thuộc vào thông tin chứa trong file
- HDH có cần biết và hỗ trợ các kiểu cấu trúc file?
- Hỗ trợ cấu trúc file ở mức HDH:
 - Ưu điểm:
 - Các thao tác với file sẽ dễ dàng hơn đối với người lập trình ứng dụng
 - HDH có thể kiểm soát được các thao tác với file
 - Nhược điểm:
 - Tăng kích thước hệ thống
 - Tính mềm dẻo của HDH bị giảm
- Thực tế các HDH chỉ coi file là tập hợp các byte không cấu trúc

I. CÁC KHÁI NIỆM

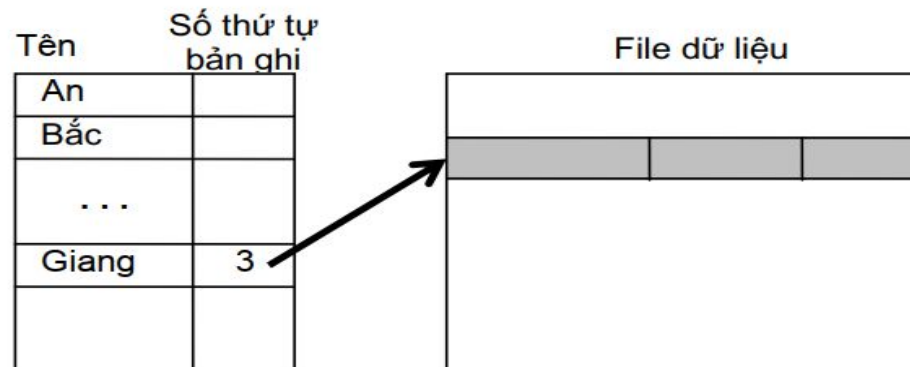
- Đa số HDH không hỗ trợ và quản lý kiểu cấu trúc file
- Cấu trúc file do chương trình ứng dụng và người dùng tự quản lý
- Trong HDH UNIX, DOS, WINDOWS, file được xem như tập hợp các byte.
- Các chương trình ứng dụng khác nhau sẽ tự tạo ra và quản lý cấu trúc file riêng mình.
- Ví dụ: chương trình đồ họa lưu file dưới dạng mã nhị phân đã được giải nén, chương trình hệ thống quản lý dữ liệu sẽ tạo ra file bao gồm các bản ghi.

- Để đọc/ghi file hệ điều hành phải quy định cách thức truy cập tới nội dung file. Mỗi HDH có thể hỗ trợ một hoặc nhiều cách truy cập khác nhau.
- Truy cập tuần tự:
 - Thông tin được đọc, ghi theo từng byte/ bản ghi lần lượt từ đầu file
 - Sử dụng 1 con trỏ để định vị vị trí hiện thời trong file
 - Sau mỗi thao tác đọc hoặc ghi con trỏ sẽ được tăng lên 1 để trỏ tới vị trí tiếp theo
 - Kiểu truy cập này rất phù hợp với các file được lưu trữ trên băng từ. Thông tin được ghi hoặc đọc lần lượt theo chiều quay của băng.

- Truy cập trực tiếp:
 - File được xem như các khối/ bản ghi được đánh số
 - Các khối có thể truy cập theo thứ tự bất kỳ
 - Chẳng hạn ta có thể đọc khối 50 sau đó đọc khối 13 rồi khối 101.
 - Việc truy cập trực tiếp dựa trên đặc điểm của đĩa cho phép truy cập các khối bất kỳ
 - File được chứa trong các khối khác nhau của đĩa do vậy cũng cho phép truy cập không tuần theo thứ tự

II. CÁC PHƯƠNG PHÁP TRUY CẬP FILE

- Truy cập dựa trên chỉ số:
 - Cho phép truy cập tới bản ghi trong file, không theo số thứ tự hoặc vị trí của bản ghi trong file mà theo khóa ứng với bản ghi đó.
 - File chứa 1 chỉ số riêng: gồm các khóa và con trỏ chỉ tới các bản ghi trong file
 - Truy cập: tìm khóa tương ứng trong chỉ mục, sau đó theo con trỏ xác định bản ghi và truy cập trực tiếp tới nó

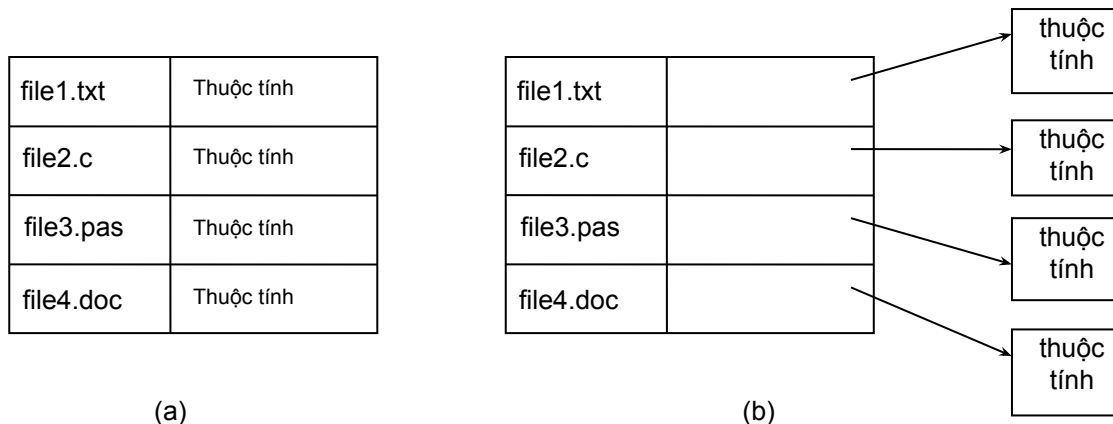


- Tạo file:
 - Tạo file trống chưa có data; được dành 1 chỗ trong thư mục kèm theo một số thuộc tính như thời gian tạo file, tên file, người tạo file,...
- Xóa file:
 - Giải phóng không gian mà dữ liệu của file chiếm trên đĩa
 - Giải phóng chỗ của file trong thư mục
 - Việc giải phóng không gian có thể đơn thuần là đánh dấu không gian như không gian tự do.
- Mở file:
 - Thực hiện trước khi ghi và đọc file
 - Đọc các thuộc tính của file trên đĩa vào bộ nhớ để tăng tốc độ cho thao tác đọc ghi tiếp theo.

- Đóng file:
 - Xóa các thông tin về file ra khỏi bảng trong bộ nhớ để nhường chỗ cho các file sắp mở.
 - Rất nhiều hệ điều hành hạn chế số lượng file được mở cùng một lúc nên việc đóng các file đã truy cập xong là rất quan trọng.
- Ghi vào file
- Đọc file

- Số lượng file lưu trữ trên đĩa rất lớn $\Rightarrow$ phải tổ chức để dễ dàng quản lý, truy cập files
- Không gian trên đĩa được chia thành các phần (partition/volume) gọi là đĩa logic
 - Ví dụ: trên máy tính PC có 1 ổ cứng và có thể chia thành các ổ C,D,E $\Rightarrow$ đĩa logic
- Để quản lý file trên các đĩa logic, thông tin về file được lưu trong thư mục của đĩa
- Thư mục = $\sum$ các khoản mục $\sim$ files
- Khoản mục chứa các thông tin về file: tên, kích thước, vị trí, kiểu file,... hoặc con trỏ tới nơi lưu trữ thông tin này
- Cõi thư mục như 1 bảng, mỗi dòng là khoản mục ứng với 1 file

- Các cách lưu thông tin về file trong thư mục:
 - Toàn bộ thuộc tính của file được lưu trong thư mục, file chỉ chứa data => kích thước khoản mục, thư mục lớn
 - Một phần thuộc tính được lưu trữ luôn cùng với dữ liệu của file. Thư mục chỉ lưu thông tin tối thiểu cần thiết cho việc tìm kiếm vị trí file trên đĩa => kích thước giảm



Hình 4.3: Lưu trữ thuộc tính file (a) Trong thư mục; (b) cùng với file

IV. THƯ MỤC

2. Các thao tác với thư mục

- Mở file:
 - HDH tìm trong thư mục khoản mục ứng với tên file cần mở
 - Đọc các thuộc tính và vị trí dữ liệu của file vào bảng chứa thông tin về các file đang mở
 - Nếu khoản mục trỏ tới CTDL khác chứa thuộc tính file, cấu trúc này sẽ được đọc vào bảng

IV. THƯ MỤC

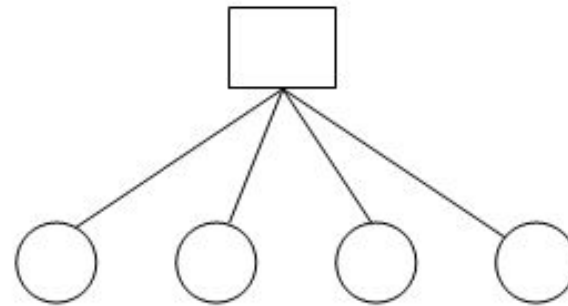
2. Các thao tác với thư mục

- **Tìm kiếm file:** cấu trúc thư mục phải cho phép tìm kiếm file theo tên file
- **Tạo file:** tạo khoản mục mới và thêm vào thư mục
- **Xóa file:** thông tin về file và khoản mục tương ứng bị xóa khỏi thư mục
- **Duyệt thư mục:** liệt kê các file trong thư mục và thông tin chứa trong khoản mục của file
- **Đổi tên file:** chỉ cần thực hiện với thư mục chứ không liên quan đến dữ liệu của file

IV. THƯ MỤC

3. Cấu trúc hệ thống thư mục

- Thư mục 1 mức:
 - Đơn giản nhất
 - Chỉ có 1 thư mục duy nhất và tất cả các file được giữ trong thư mục này
 - Khó chọn tên cho file
 - Tìm kiếm file khó



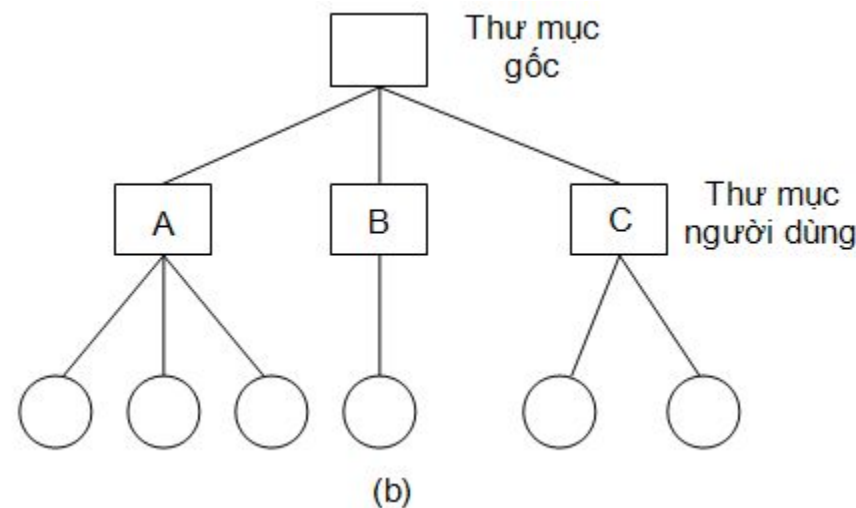
(a)

 = Thư mục  = File

IV. THƯ MỤC

3. Cấu trúc hệ thống thư mục

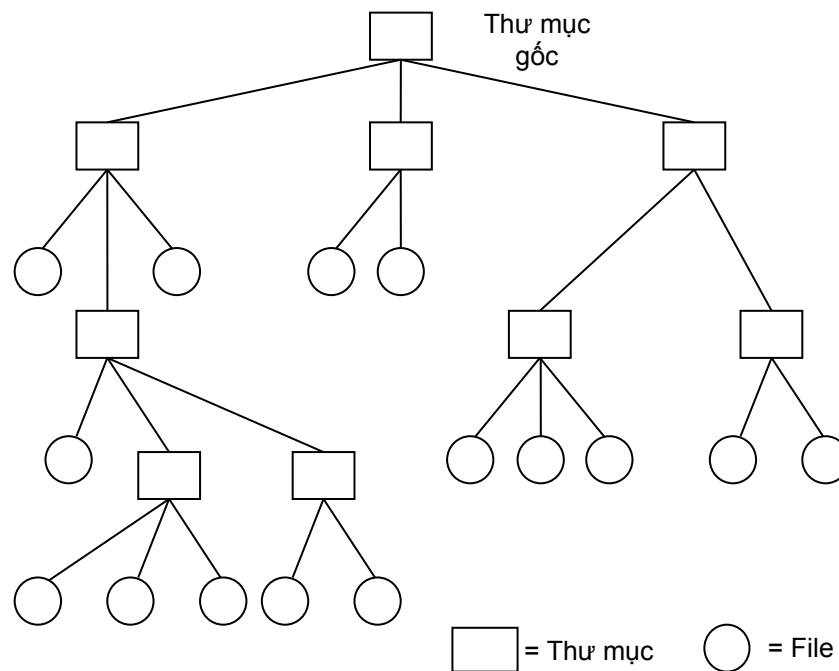
- Thư mục 2 mức:
 - Phân cho mỗi người dùng 1 thư mục riêng (UFD: User File Directory), chứa các file của mình
 - Khi người dùng truy cập file, file sẽ được tìm kiếm trong thư mục ứng với tên người đó
 - => các người dùng khác nhau có thể đặt tên file trùng nhau
- Cô lập người dùng
- Các file mà nhiều người dùng truy cập tới => chép vào từng thư mục của từng người dùng => lãng phí



IV. THƯ MỤC

3. Cấu trúc hệ thống thư mục

- Thư mục cấu trúc cây:
 - Thư mục con có thể chứa các thư mục con khác và các files
 - Hệ thống thư mục được biểu diễn phân cấp như 1 cây: cành là thư mục, lá là file



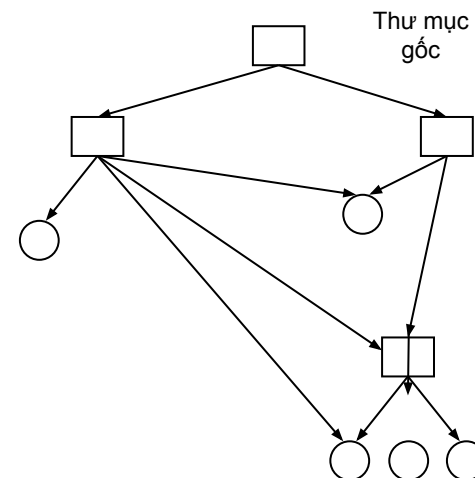
IV. THƯ MỤC

3. Cấu trúc hệ thống thư mục

- Thư mục cấu trúc cây (tt):
 - Phân biệt khoản mục file và khoản mục của thư mục con: thêm bit đặc biệt trong khoản mục
 - 1: khoản mục của thư mục mức dưới
 - 0: khoản mục của file
 - Tại mỗi thời điểm, người dùng làm việc với thư mục hiện thời (current directory)
 - Tổ chức cây thư mục cho từng đĩa:
 - Trong hệ thống file như FAT của DOS, cây thư mục được xây cho từng đĩa. Hệ thống thư mục được coi là rừng, mỗi cây trên 1 đĩa
 - Linux: toàn hệ thống chỉ gồm 1 cây thư mục

3. Cấu trúc hệ thống thư mục

- Thư mục cấu trúc đồ thị không tuần hoàn (acyclic graph):
 - Chia sẻ files và thư mục để có thể xuất hiện ở nhiều thư mục riêng khác nhau
 - Đây là một cấu trúc đồ thị không chứa chu trình, là mở rộng của cấu trúc cây
 - cây: lá và cành có thể đồng thời thuộc về những cành khác nhau
 - Triển khai:
 - Sử dụng liên kết: con trỏ tới thư mục hoặc file khác
 - Tạo bản sao của file và thư mục cần chia sẻ và chứa vào các thư mục khác nhau => phải đảm bảo tính đồng bộ và nhất quán => tránh sao lưu chồng chéo, thống kê một file nhiều lần.
 - Mềm dẻo nhưng phức tạp



- Mô tả vị trí của file trong thư mục
- Đường dẫn tuyệt đối:
 - Đường dẫn từ gốc của cây thư mục, đi qua các thư mục trung gian, dẫn tới file
 - C:\bc\bin\bc.exe
- Đường dẫn tương đối:
 - Tính từ thư mục hiện thời
 - Thêm 2 khoản mục đặc biệt trong thư mục: “.” để biểu diễn thư mục hiện tại, và “..” cho thư mục mức trên.

IV. THƯ MỤC

5. Tổ chức bên trong của thư mục

- Danh sách:
 - Tổ chức thư mục dưới dạng danh sách các khoản mục
 - Tìm kiếm khoản mục được thực hiện bằng cách duyệt lần lượt danh sách
 - Thêm file mới vào thư mục:
 - Duyệt cả thư mục để kiểm tra xem khoản mục với tên file như vậy đã có chưa
 - Khoản mục mới được thêm vào cuối danh sách hoặc 1 ô trong bảng
 - Mở file, xóa file
 - Tìm kiếm trong danh sách chậm
 - Lưu trữ thư mục trong bộ nhớ

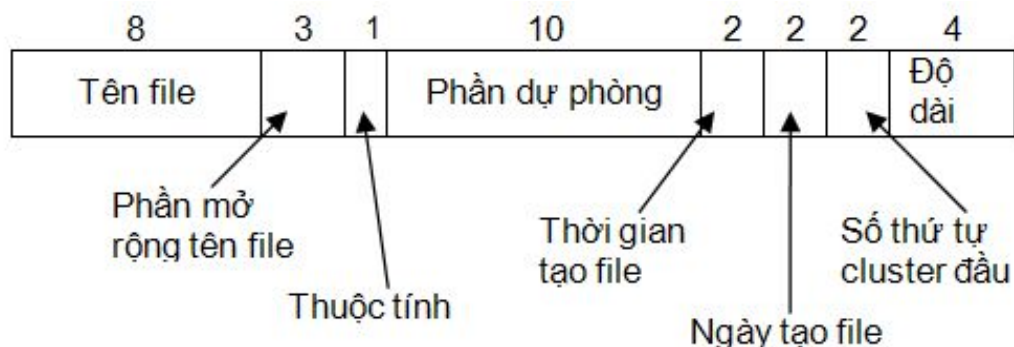
IV. THƯ MỤC

5. Tổ chức bên trong của thư mục

- Cây nhị phân:
 - Tăng tốc độ tìm kiếm nhờ CTDL có hỗ trợ sắp xếp
 - Hệ thống file NTFS của WinNT
- Bảng băm (hash table):
 - Dùng hàm băm để tính vị trí của khoản mục trong thư mục theo tên file
 - Thời gian tìm kiếm nhanh
 - Hàm băm phụ thuộc vào kích thước của bảng băm => kích thước bảng cố định

5. Tổ chức bên trong của thư mục

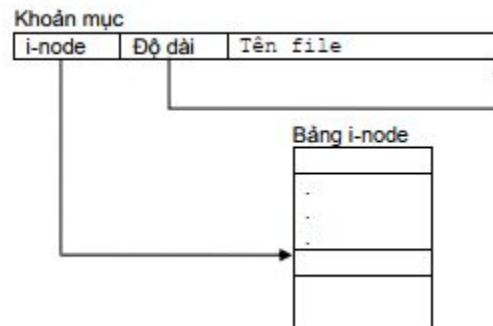
- Tổ chức thư mục của DOS:
 - Mỗi đĩa logic có cây thư mục riêng, bắt đầu từ thư mục gốc **ROOT**
 - Thư mục gốc được đặt ở phần đầu của đĩa, ngay sau sector khởi động **BOOT** và bảng **FAT**
 - Thư mục gốc chứa files và các thư mục con
 - Thư mục con có thể chứa files và các thư mục cấp dưới nữa
 - Được tổ chức dưới dạng bảng: mỗi khoản mục chiếm 1 dòng trong bảng và có kích thước cố định 32 bytes



IV. THƯ MỤC

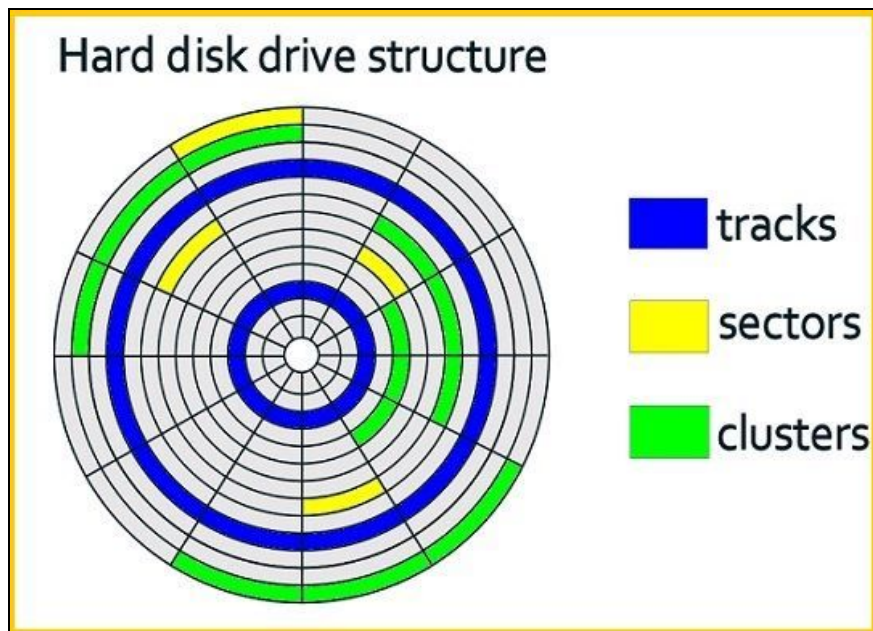
5. Tổ chức bên trong của thư mục

- Tổ chức thư mục của Linux:
 - Thư mục hệ thống file Ext2 của Linux có cách tổ chức đơn giản
 - Khoản mục chứa tên file và địa chỉ I-node
 - Thông tin còn lại về các thuộc tính file và vị trí các khối dữ liệu được lưu trên I-node chứ không phải thư mục
 - Kích thước khoản mục phụ thuộc vào độ dài tên file
 - Phần đầu của khoản mục có trường cho biết kích thước khoản mục



V. CẤP PHÁT KHÔNG GIAN CHO FILE

- Nhiệm vụ quan trọng của HĐH trong việc quản lý hệ thống file là cấp phát không gian trên đĩa và các thiết bị nhớ ngoài khác để lưu trữ file và thư mục
- Đồng thời, ghi lại vị trí các khối nhớ đã cấp phát để có thể tiến hành truy cập về sau.



Tổ chức của đĩa cứng

- ❖ Sector là đơn vị nhỏ nhất do chương trình điều khiển đĩa cho phép đọc hoặc ghi (khối vật lý) 1 sector=512byte
- ❖ Cluster gồm một số khối vật lý, là đơn vị thông tin nhỏ nhất mà hệ điều hành cấp phát cho file (khối logic) 2KB-32KB

V. CẤP PHÁT KHÔNG GIAN CHO FILE

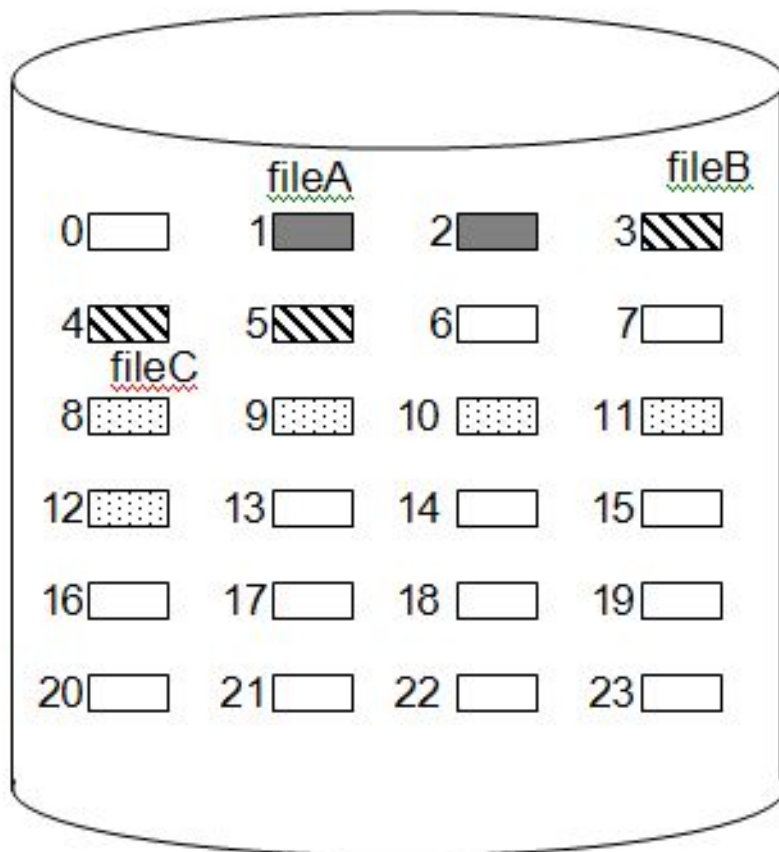
- Mỗi file gồm 1 tập các khối, HĐH chịu trách nhiệm cấp phát các khối cho file:
 - Không gian trên đĩa phải được cấp phát cho file
 - Cần theo dõi không gian trống sẵn sàng cho việc cấp phát
 - Cần thực hiện sao cho tiết kiệm không gian trên đĩa và tăng tốc độ truy cập file
- Một số vấn đề:
 - Không gian tối đa yêu cầu cấp phát cho file 1 lần là bao nhiêu?
 - Không gian cấp phát cho file gọi là phần (partion). Kích thước phần ntn?

1. Cấp phát các khối liên tiếp

- Mỗi file được cấp phát 1 khoảng không gian gồm các khối liên tiếp trên đĩa
- Vị trí file trên đĩa được xác định bởi vị trí khối đầu tiên và độ dài (số khối) mà file đó chiếm
- Khi có yêu cầu cấp phát, HDH sẽ chọn 1 vùng trống có số lượng khối đủ cấp cho file đó
- Bảng cấp phát file chỉ cần 1 khoản mục cho 1 file, chỉ ra khối bắt đầu, và độ dài của file tính = khối
- Là cấp phát trước, sử dụng kích thước phần thay đổi

1. Cấp phát các khối liên tiếp (tt)

External fragmentation

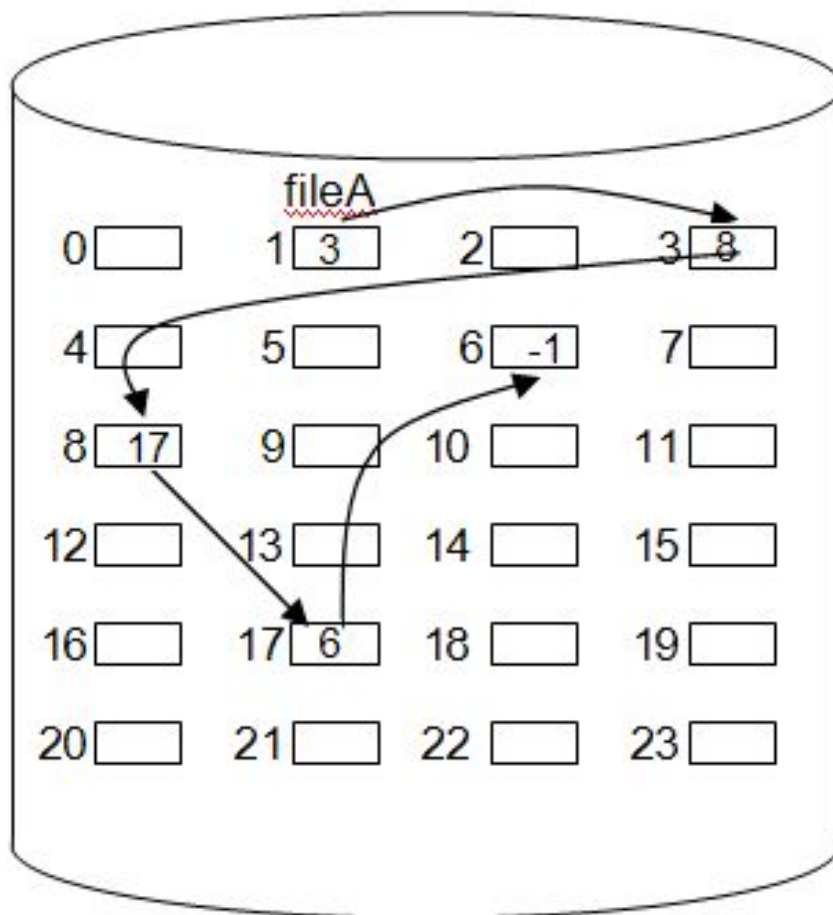


Tên file	Thư mục	
	Bắt đầu	Độ dài
fileA	1	2
fileB	3	3
fileB	8	5

■ Ưu điểm: 1. Cấp phát các khối liên tiếp (tt)

- Cho phép truy cập trực tiếp và tuần tự
- Đơn giản, tốc độ cao do tiết kiệm thời gian di chuyển đầu từ khi đọc các khối của file
- Nhược điểm:
 - Phải biết trước kích thước file khi tạo
 - Khó tìm ra khoảng không gian trống đủ lớn trên đĩa để cấp phát cho file
 - Sau thời gian sử dụng, các khối được cấp phát khi tạo file và giải phóng file khi xóa file sẽ tạo rãnh trống trên đĩa => khi có yêu cầu cấp phát hdh cần kiểm tra vùng trống.
 - Gây phân mảnh ngoài: đó là hiện tượng vùng trống còn lại trên đĩa có kích thước quá nhỏ do vậy không thể cấp phát cho file có kích thước lớn hơn.

- Các khối thuộc về mỗi file được kết nối với nhau thành danh sách kết nối; phần đầu mỗi khối chứa con trỏ trỏ tới khối tiếp theo
- Các khối thuộc về 1 file có thể nằm ở vị trí bất kì trên đĩa
- Khoản mục của thư mục chứa con trỏ tới khối đầu tiên của file
- Khi file được cấp thêm khối mới, khối đó được thêm vào cuối danh sách
- HDH đọc lần lượt từng khối và sử dụng con trỏ để xác định khối tiếp theo



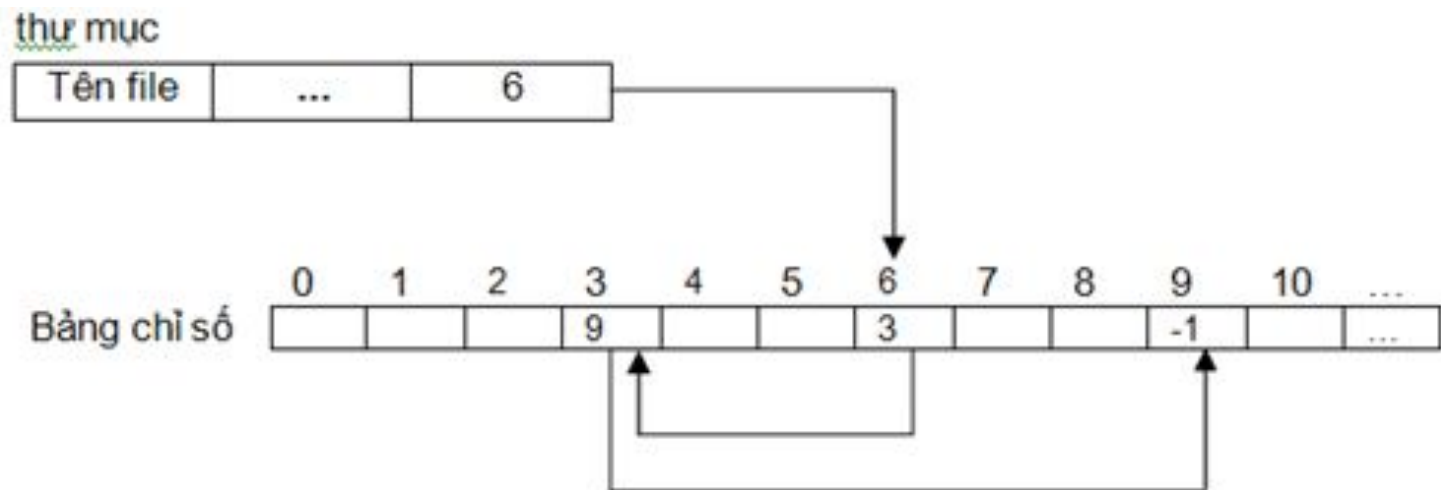
Thư mục	
Tên file	Bắt đầu
fileA	1

- Ưu điểm:
 - Do các khối thuộc về một file có thể nằm bất cứ chỗ nào trên đĩa, không nhất thiết phải nằm cạnh nhau
- => Không bị phân mảnh ngoài
 - Không yêu cầu biết trước kích thước file lúc tạo
 - Dễ tìm vị trí cho file, khoản mục đơn giản

- Nhược điểm:
 - Để đọc một khối nào đó ta phải theo tất cả các con trỏ từ khối đầu tiên cho tới khối cần đọc => Không hỗ trợ truy cập trực tiếp.
 - Con trỏ nằm ngay trong khối và khối là đơn vị nhỏ nhất có thể tiến hành đọc nên để xác định con trỏ, ta phải đọc cả khối => để đọc một khối ta phải đọc từ khối đầu tiên cho tới khối cần đọc.
 - Trong trường hợp giá trị của con trỏ thay đổi => xác định khối nào thuộc file nào sẽ không chính xác => giảm độ tin cậy và tính toàn vẹn của hệ thống file
 - Tốc độ truy cập không cao

3. Sử dụng danh sách kết nối trên bảng chỉ số

- Bảng chỉ số: mỗi ô của bảng ứng với 1 khối của đĩa
- Con trỏ tới khối tiếp theo của file được chứa trong ô tương ứng của bảng
- Mỗi đĩa logic có 1 bảng chỉ số được lưu ở vị trí xác định
- Kích thước mỗi ô trên bảng phụ thuộc vào số lượng khối trên đĩa

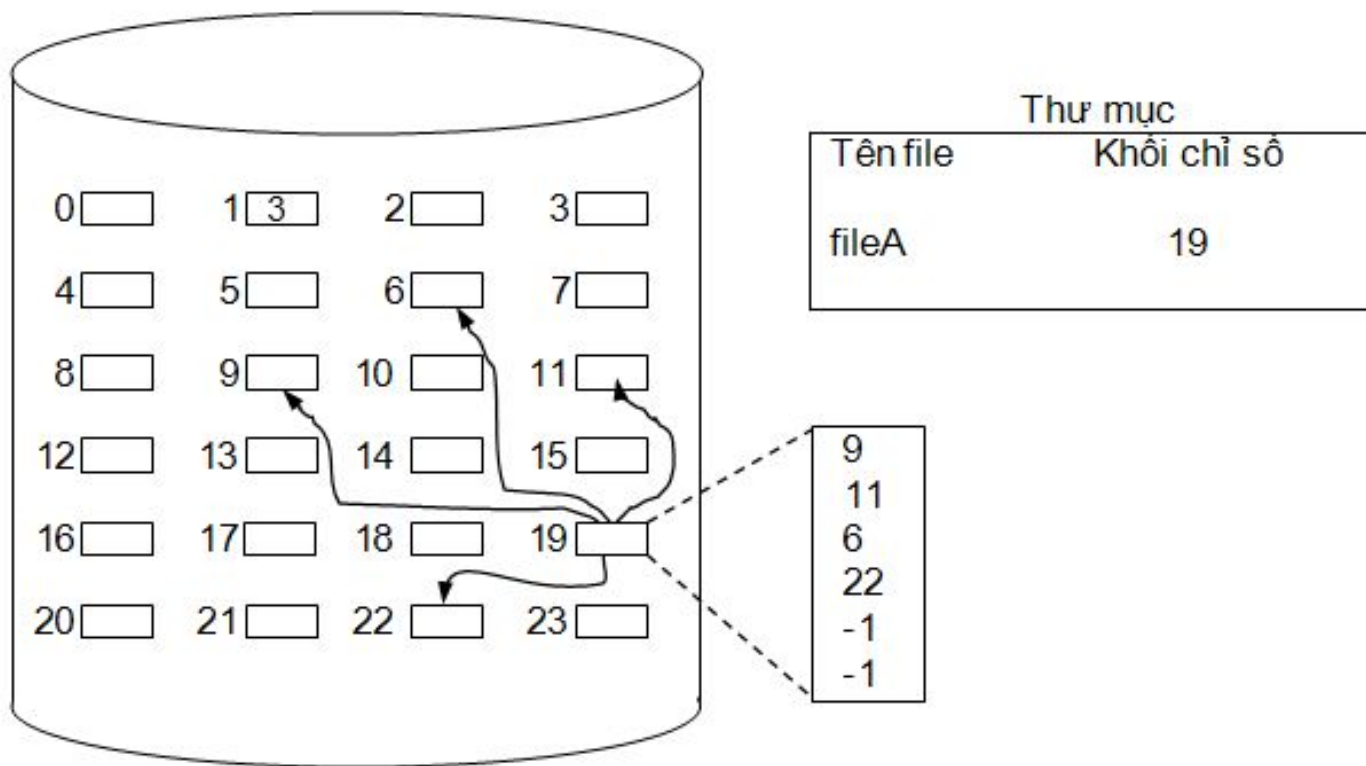


V. CẤP PHÁT KHÔNG GIAN CHO FILE

3. Sử dụng danh sách kết nối trên bảng chỉ số (tt)

- Cho phép tiến hành truy cập file trực tiếp: đi theo chuỗi con trỏ chứa trong bảng chỉ mục
- Bảng FAT (File Allocation Table): được lưu ở đầu mỗi đĩa logic sau sector khởi động
- FAT12, FAT16, FAT32: mỗi ô của bảng có kích thước 12, 16, 32 bit cho phép quản lý tối đa 2^{12} , 2^{16} , 2^{32}

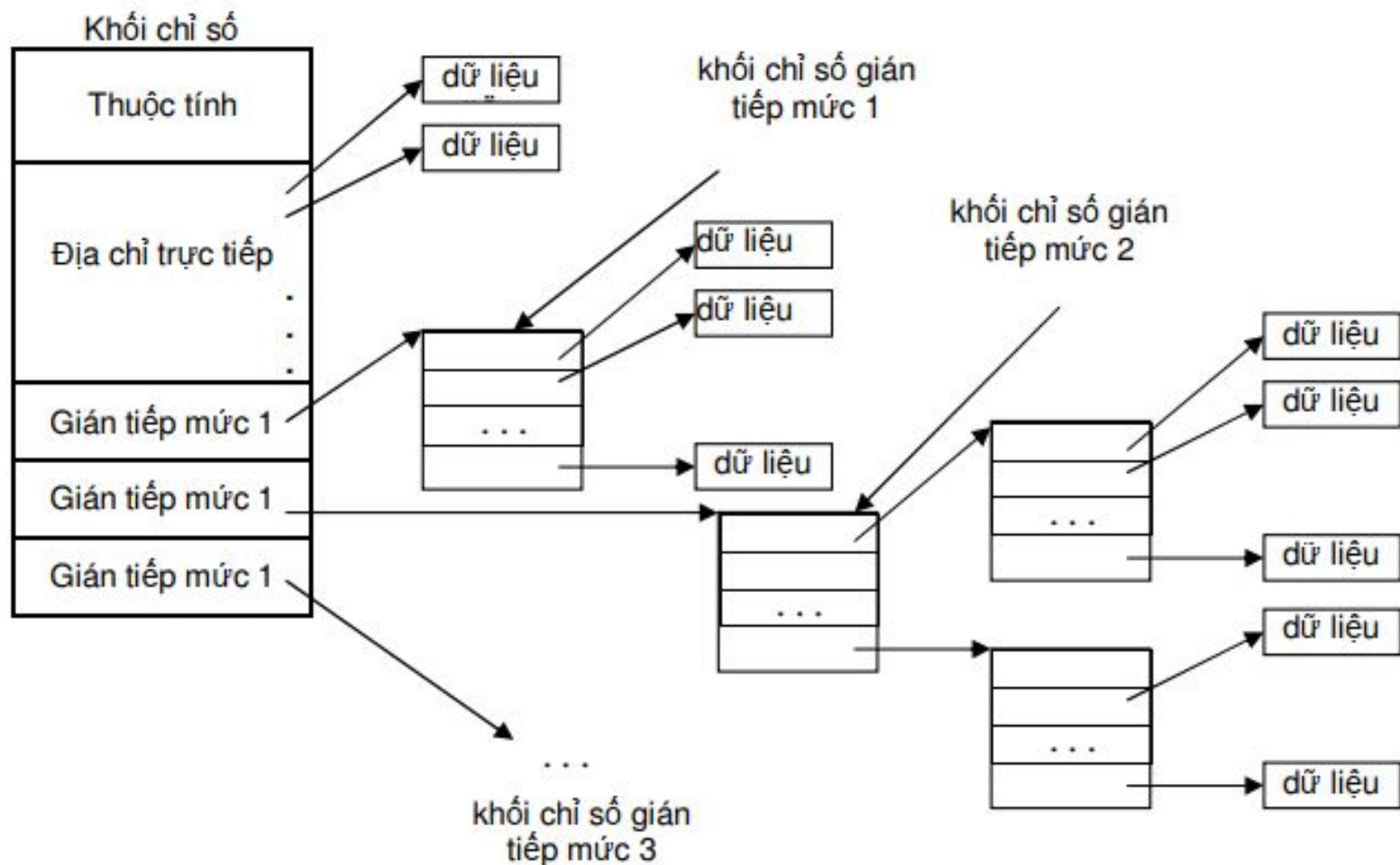
- Tất cả con trỏ tới các khối thuộc về 1 file được tập trung 1 chỗ
- Mỗi file có một mảng riêng của mình chứa trong một khối gọi là khối chỉ mục (I-node)
- Mảng chứa thuộc tính của file và vị trí các khối của file trên đĩa
- Ô thứ i của mảng chứa con trỏ tới khối thứ i của file
- Khoản mục của file trong thư mục chứa con trỏ tới khối chỉ mục này



- Chọn kích thước I-node:
 - Nhỏ: tiết kiệm không gian nhưng không đủ con trỏ tới các khối nếu file lớn
 - Lớn: với file nhỏ chỉ chiếm 1 vài ô thì lãng phí
 - Giải pháp:
 - Thay đổi kích thước i-node = sử dụng danh sách kết nối
 - Sử dụng I-node có cấu trúc nhiều mức

- Ưu điểm:
 - Cho phép truy cập trực tiếp
 - Các khối thuộc 1 file không cần nằm liên tiếp nhau
- Nhược điểm:
 - Tốc độ truy cập file chậm

4. Sử dụng khối chỉ mục (index block/ node)



- Không gian trên đĩa được cấp phát theo từng khối không phân chia. Để quản lý hiệu quả không gian trên đĩa, ngoài phương pháp cấp phát khối nhớ cho file, cần chú ý:
 - Lựa chọn kích thước cho khối
 - Quản lý khối trống
- Kích thước khối:

Khối là đơn vị cấp phát nhỏ nhất của hệ điều hành, mỗi file bao gồm một số nguyên các khối nhớ.

- Kích thước khối lớn:
 - Giảm kích thước bảng chỉ mục, tăng tốc độ đọc file;
 - Bị phân mảnh trong
- Kích thước khối nhỏ:
 - Mỗi file chiếm nhiều khối nhớ, nằm rải rác trên đĩa
 - Thời gian đọc file lâu
- Chọn kích thước khối tùy thuộc:
 - Kích thước đĩa: đĩa lớn, chọn kích thước khối lớn => thời gian truy cập nhanh, đơn giản hóa việc quản lý
 - Kích thước file: hệ thống sử dụng nhiều file lớn, kích thước tăng và ngược
- Kích thước khối thường là lũy thừa 2 của sector và nằm trong khoảng từ 512B tới 32 KB

- ❖ Để có thể cấp phát khối nhớ trên đĩa cho file, HDH cần biết khối nào hiện đang trống. Các khối trống bao gồm những khối chưa được cấp phát lần nào hoặc cấp phát rồi nhưng đã được giải phóng.

1. Bảng bit

- Vector bit là mảng 1 chiều
- Mỗi ô có kích thước 1 bit tương ứng với một khối trên đĩa
- Khối được cấp phát có bit tương ứng là 0, khối trống: 1 hoặc ngược lại
- Dễ tìm 1 hoặc nhóm các khối trống liên tiếp
- Với đĩa có kích thước lớn, đọc toàn bộ vector bit vào MEM có thể đòi hỏi khá nhiều không gian nhớ

1. Bảng bit

- Ví dụ: Trong trường hợp sau, các khối 0, 1, 8, 9, 10 đã được cấp phát, các khối còn lại chưa bị sử dụng:

0011111100011111...

- Các khối trống được liên kết với nhau thành danh sách
- Mỗi khối trống chứa con trỏ chỉ tới khối trống tiếp theo
- Địa chỉ khối trống đầu tiên được lưu ở vị trí đặc biệt trên đĩa và được HDH giữ trong MEM khi cần làm việc với các file
- Đòi hỏi truy cập lần lượt khi cần duyệt danh sách này
- HDH có thể cấp phát ngay các khối ở đầu danh sách

- Các khối nằm liền nhau thường được cấp phát và giải phóng đồng thời
- Lưu vị trí khối trồng đầu tiên của vùng các khối trồng liên tiếp và số lượng các khối trồng nằm liền sau đó
- Thông tin trên được lưu vào danh sách riêng

- File là nơi lưu trữ thông tin rất quan trọng đối với người dùng, thậm chí quan trọng với chính bản thân hệ thống tính toán.
- Hệ thống file phải có tính bền vững cao, phải đảm bảo lưu trữ thông tin sao cho thông tin được toàn vẹn và không bị mất mát.
- Thông tin của hệ thống file lưu trên đĩa và các thiết bị nhớ ngoài khác có thể bị sai lệch, mất mát do:
 - Lỗi phần cứng/phần mềm
 - Lỗi sự cố kỹ thuật
 - Mặc dù không thể ngăn chặn các sự cố như vậy => cũng cần được xây dựng sao cho có khả năng phát hiện và khắc phục hậu quả trong khả năng cao nhất

- Trong quá trình sản xuất cũng như sử dụng, đĩa có thể chứa các khối hỏng, không thể dùng cho đọc/ghi thông tin.
 - Đĩa mềm, các khối hỏng thường xuất hiện trong quá trình sử dụng
 - Đĩa cứng: thường có các khối hỏng ngay trong quá trình sản xuất. Những khối hỏng này được nhà sản xuất phát hiện trong quá trình kiểm tra đĩa và được lưu trong danh sách khối hỏng đi cùng với đĩa

- Phát hiện và loại trừ khối hỏng
 - Phương pháp 1:
 - Một sector trên đĩa được dành riêng chứa danh sách các khối hỏng
 - Một số khối không hỏng được dành riêng để dự trữ
 - Các khối hỏng sẽ được thay thế bởi các khối dự trữ bằng cách thay thế địa chỉ
 - Truy cập tới khối hỏng thành truy cập tới khối dự trữ
 - Phương pháp 2:
 - Tập trung tất cả các khối hỏng thành 1 file
 - => được coi như đã cấp phát và không được sử dụng nữa

2. Sao dự phòng

- Tạo ra một bản sao của đĩa trên một vật mang khác
- Sao lưu toàn bộ (full backup):
 - Ghi toàn bộ thông tin trên đĩa ra vật mang tin khác
 - Chắc chắn nhưng tốn nhiều thời gian
- Sao lưu tăng dần (incremental backup):
 - Được sử dụng sau khi đã tiến hành full backup ít nhất 1 lần
 - Chỉ ghi lại các file đã bị thay đổi sau lần sao lưu cuối cùng
 - Hệ thống lưu trữ thông tin về các lần lưu trữ file
 - DOS: file thay đổi, archive bit = 1
- Kết hợp:
 - Full backup: hàng tuần/ tháng
 - Incremental backup: hàng ngày

- Hệ thống file chứa nhiều CTDL có mối liên kết => thông tin về liên kết bị hư hại, tính toàn vẹn của hệ thống bị phá vỡ => đứt đoạn, không biết lưu ở khối nào tiếp theo
- Các khối không có mặt trong danh sách các khối trống, đồng thời cũng không có mặt trong một file nào
- Một khối có thể vừa thuộc về một file nào đó vừa có mặt trong danh sách khối trống
- HDH có các chương trình kiểm tra tính toàn vẹn của hệ thống file, được chạy khi hệ thống khởi động, đặc biệt là sau sự cố
- Ví dụ: chương trình SCANDISK củ WIN hoặc DOS

- Ví dụ trong hệ UNIX:
 - Tạo hai số đếm cho mỗi khối:
 - Số đếm thứ nhất: số lần khối đó xuất hiện trong danh sách khối trống.
 - Số đếm thứ hai: số lần khối xuất hiện trong file
 - Tất cả số đếm được khởi tạo bằng 0
 - Duyệt danh sách khối trống và toàn bộ i-node của các file
 - Một khối xuất hiện trong danh sách khối trống, số đếm tương ứng thứ nhất được tăng một đơn vị
 - Nếu khối xuất hiện trong i-node của file, số đếm tương ứng thứ hai được tăng một đơn vị
 - Tổng 2 số đếm = 1

Số lần xuất hiện trong danh sách trống

Số thứ tự khối					
0	1	2	3	4	5
0	1	0	1	0	1

Số lần xuất hiện trong file

Số thứ tự khối					
0	1	2	3	4	5
0	0	1	1	3	2

- Mặc dù có thể kiểm tra tính toàn vẹn của các liên kết trong hệ thống file. Tuy nhiên, kiểm tra toàn bộ hệ thống file đòi hỏi nhiều thời gian. Việc kiểm tra chỉ cho phép phát hiện lỗi sau khi đã xảy ra và không đảm bảo khôi phục dữ liệu đối với một số lỗi.
- Giao tác (transaction) là một tập hợp các thao tác cần phải được thực hiện trọn vẹn cùng với nhau
- Với hệ thống file: mỗi giao tác sẽ bao gồm những thao tác thay đổi liên kết cần thực hiện cùng nhau (Ví dụ các thao tác cập nhật liên kết liên quan với một khối trên đĩa)

- Toàn bộ trạng thái hệ thống file được ghi lại trong file log
- Mỗi khi thực hiện giao tác, hệ thống kiểm tra xem giao tác có được thực hiện trọn vẹn không.
- Nếu giao tác không được thực hiện trọn vẹn, HDH sử dụng thông tin từ log để khôi phục hệ thống file về trạng thái không lỗi trước khi thực hiện giao tác

- Ngăn cản việc truy cập trái phép các thông tin lưu trữ trong file và thư mục
- Đối với hệ thống nhỏ dành cho một người dùng vấn đề bảo mật tương đối đơn giản, có thể thực hiện bằng các biện pháp vật lý => Không cho người khác tiếp cận tới hệ thống,
- Trong những hệ thống tính toán đa người dùng, việc bảo mật cho file và thư mục thực hiện bằng:
 - Kiểm soát các thao tác truy cập tới file hoặc thư mục
 - Dùng mật khẩu => Người dùng phải nhớ nhiều mật khẩu => Mỗi khi thao tác với tài nguyên lại gõ mật khẩu

- Sử dụng danh sách quản lý truy cập ACL (Access Control List)
 - Mỗi file được gán danh sách đi kèm, chứa thông tin định danh người dùng và các quyền người đó được thực hiện với file
 - ACL thường được lưu trữ như thuộc tính của file/ thư mục
 - Thường được sử dụng cùng với cơ chế đăng nhập
 - Các quyền truy cập cơ bản:
 - Quyền đọc (r): người có quyền này được phép đọc nội dung file
 - Quyền ghi, thay đổi (w): được phép ghi vào file, tức là thay đổi nội dung file
 - Quyền xóa: được phép xóa file, quyền này tương đương với quyền thay đổi file
 - Quyền thay đổi chủ file (change owner)

- Sử dụng danh sách quản lý truy cập ACL (Access Control List)

File 1

A	B	C
chủ file		
đọc	đọc	đọc
ghi		ghi

File 2

B	C
chủ file	
đọc	đọc
ghi	

IX. HỆ THỐNG FILE FAT

- Hệ thống FAT được thiết kế để sử dụng trong HDH DOS, sau đó đã được sử dụng trong một số phiên bản của HDH Window 3.0, 3.1, 95/98.
- Hiện nay, FAT vẫn là hệ thống thông dụng, được sử dụng trong hầu hết các HDH hiện nay để quản lý thẻ nhớ, đĩa mềm, đĩa CD
- 3 phiên bản: FAT12, FAT16, FAT32
- Chữ số chỉ kích thước ô bảng FAT tương ứng 12, 16 và 32 bit
- Hiện nay, FAT32 thường được sử dụng cho đĩa cứng, FAT16 được sử dụng cho thiết bị nhớ ngoài có dung lượng nhỏ như CD, thẻ nhớ ngoài.

IX. HỆ THỐNG FILE FAT

1. ĐĨA LOGIC

- Đơn vị tổ chức trong hệ thống file FAT là đĩa logic.
- Đơn vị cấp phát không gian trên đĩa (khối logic) là các **cluster**.
Mà mỗi cluster chứa một số lượng sector bằng lũy thừa của 2.
- Thông tin trên đĩa logic được tổ chức:

Boot sector và các khối dự phòng	Bảng FAT1	Bảng FAT2	Thư mục gốc (chỉ có trên FAT12 và FAT16)	Phần còn lại cho tới cuối đĩa chứa các file và thư mục của đĩa lô gic
-------------------------------------	-----------	-----------	---	--

▪ Boot sector:

- Sector đầu tiên của đĩa logic
- Chứa thông tin mô tả cấu trúc đĩa logic: kích thước sector, cluster, kích thước bảng FAT
- Chứa mã chương trình môi để tải HĐH nếu đĩa logic là đĩa khởi động

IX. HỆ THỐNG FILE FAT

1. ĐĨA LOGIC

Boot sector và các khối dự phòng	Bảng FAT1	Bảng FAT2	Thư mục gốc (chỉ có trên FAT12 và FAT16)	Phần còn lại cho tới cuối đĩa chứa các file và thư mục của đĩa lô gic
-------------------------------------	-----------	-----------	--	--

- **FAT:** bảng chỉ số quản lý cấp phát khối cho file. Do thông tin vùng này rất quan trọng nên HDH thường duy trì hai bảng FAT, trong đó một bảng là bản sao của bảng kia. Số lượng FAT có thể thay đổi và được ghi ở Boot sector.
- **Thư mục gốc ROOT:** vùng này chứa các thư mục gốc, tức là thư mục trên cùng của đĩa. Kích thước vùng này bằng số lượng tối đa khoản mục trong thư mục gốc và được ghi trong boot sector. FAT32 không dùng vùng này mà lưu trữ thư mục gốc như các file trong thư mục thông thường.
- **Vùng dữ liệu:** chứa các file và thư mục của đĩa logic

IX. HỆ THỐNG FILE FAT

BOOT SECTOR

Boot sector là sector đầu tiên trên đĩa và chứa thông tin mô tả cấu trúc đĩa cùng với mã để tài HDH đối với đĩa mỗi HDH. Cấu trúc cụ thể của boot sector như sau: 32 Byte đầu tiên

Vị trí	Độ dài	Ý nghĩa
0	3	Lệnh Jump. Chỉ thị cho CPU bỏ qua phần thông tin và nhảy tới thực hiện phần mã khởi của hệ điều hành nếu đây là đĩa khởi hệ điều hành.
3	8	Tên hãng sản xuất, bổ sung dấu trắng ở cuối cho đủ 8B. Ví dụ: IBM 3.3, MSDOS5.0.v.v.
11	2	Bytes per sector. Kích thước sector tính bằng byte. Giá trị thường gặp là 512 đối với đĩa cứng. Đây cũng là vị trí bắt đầu của Khối Thông số BIOS (BIOS Parameter Block, viết tắt là BPB)
13	1	Sectors per cluster. Số sector trong một cluster, luôn là lũy thừa của 2 và không lớn hơn 128.
14	2	Reserved sectors. Số lượng sector dành cho vùng đầu đĩa đến trước FAT, bao gồm boot sector và các sector dự phòng.
16	1	Số lượng bảng FAT. Thường bằng 2.
17	2	Số khoản mục tối đa trong thư mục gốc ROOT. Chỉ sử dụng cho FAT12 và FAT16. Bằng 0 với FAT32.
19	2	Total sector. Tổng số sector trên đĩa. Nếu bằng không thì số lượng sector được ghi bằng 4 byte tại vị trí 0x20.
21	1	Mô tả loại đĩa. Ví dụ 0xF0 là đĩa mềm 3.5" hai mặt với 80 rãnh trên mỗi mặt, 0xF1 là đĩa cứng .v.v.
22	2	Sectors per FAT. Kích thước FAT tính bằng sector (đối với FAT12/16)
24	2	Sectors per track. Số sector trên một rãnh.
26	2	Number of heads. Số lượng đầu đọc (mặt đĩa được sử dụng)
28	4	Hidden sectors. Số lượng sector ẩn.
32	4	Total sector. Tổng số sector trên đĩa cho trường hợp có nhiều hơn 65535.

IX. HỆ THỐNG FILE FAT BOOT SECTOR

Các byte tiếp theo với FAT12/16

Vị trí	Độ dài	Ý nghĩa
36	1	Số thứ tự vật lý của đĩa (0: đĩa mềm, 80h: đĩa cứng .v.v.)
37	1	Dự phòng
38	1	Dấu hiệu của phần mã môi. Chứa giá trị 0x29 (ký tự ‘)’) hoặc 0x28.
39	4	Số xê ri của đĩa (Volume Serial Number) được tạo lúc format đĩa
43	11	Volume Label: Nhãn của đĩa được tạo khi format.
54	8	Tên hệ thống file FAT, ví dụ "FAT12 ", "FAT16 "
62	448	Mã môi hệ điều hành, đây là phần chương trình tải hệ điều hành khi khởi động.
510	2	Dấu hiệu Boot sector (0x55 0xAA)

Các byte tiếp theo với FAT32

Vị trí	Độ dài	Ý nghĩa
36	4	Sectors per FAT. Kích thước FAT tính bằng sector.
0x28	2	Cờ của FAT
0x2a	2	Version. Phiên bản.
0x2c	4	Số thứ tự của cluster đầu tiên của thư mục gốc root.
0x30	2	Số sector của Information Sector. Đây là phần nằm trong số sector dự phòng ngay sau boot sector.
0x32	2	Số thứ tự sector đầu tiên của bản sao của boot sector (nếu có)
0x34	12	Dự phòng
0x40	1	Số thứ tự vật lý của đĩa
0x41	1	Dự phòng
0x42	1	Dấu hiệu của phần mã mỗi mở rộng.
0x43	4	Số xê ri của đĩa (Volume Serial Number)
0x47	11	Volume Label
0x52	8	"FAT32"
0x5a	420	Mã mỗi hệ điều hành
0x1FE	2	Dấu hiệu Boot sector (0x55 0xAA)

- Quản lý các cluster trên đĩa và các file theo nguyên tắc:
 - Các khối thuộc cùng 1 file được liên kết thành 1 danh sách
 - Con trỏ được chứa trong ô tương ứng của bảng FAT
- Mỗi ô trong bảng FAT tương ứng với một cluster trên đĩa, chứa 1 trong các thông tin:
 - STT cluster tiếp theo trong danh sách các khối của file
 - Dấu hiệu kết thúc nếu ô tương ứng với cluster cuối cùng của file
 - Dấu hiệu đánh dấu cluster hỏng, không được sử dụng
 - Dấu hiệu đánh dấu cluster dự phòng
 - Bằng 0 nếu cluster trống, chưa cấp phát cho file nào

IX. HỆ THỐNG FILE FAT

BẢNG FAT

- Cluster đầu tiên của vùng dữ liệu được đánh STT là 2
- 2 ô đầu tiên của bảng FAT không dùng để quản lý cluster

FAT12	FAT16	FAT32	Ý nghĩa
0x000	0x0000	0x00000000	Cluster trống
0x001	0x0001	0x00000001	Cluster dự phòng, không được sử dụng
0x002–0xFE F	0x0002–0xFFEF	0x00000002–0x0FFFFFFEF	Cluster đã được cấp cho file. Chứa số thứ tự cluster tiếp theo của file.
0xFF0–0xFF6	0xFFFF0–0xFFFF6	0xFFFFFFFF0–0xFFFFFFFF6	Cluster dự phòng
0xFF7	0xFFFF7	0xFFFFFFFF7	Cluster hỏng.
0xFF8–0xFFFF	0xFFFF8–0xFFFFF	0xFFFFFFFF8–0xFFFFFFFFF	Cluster cuối cùng của file

- Mỗi thư mục được lưu trong bảng thư mục, thực chất là 1 file đặc biệt chứa các khoản mục của thư mục
- Mỗi khoản mục chứa thông tin về một file hoặc thư mục con của thư mục đang xét
- Với FAT12/16, thư mục trên cùng của đĩa được chứa trong 1 vùng đặc biệt gọi là thư mục gốc
- Các thư mục mức thấp hơn/ thư mục gốc của FAT32 được chứa trong vùng dữ liệu trên đĩa cùng với các file
- Mỗi thư mục gồm các khoản mục 32 byte xếp liền nhau

IX. HỆ THỐNG FILE FAT

THƯ MỤC GỐC (ROOT)

Vị trí	Độ dài	Mô tả
0	8	Tên file, thêm bằng dấu trắng ở cuối nếu ngắn hơn 8 byte
8	3	Phần mở rộng, thêm bằng dấu trắng ở cuối nếu ngắn hơn 3 byte
11	1	Byte thuộc tính của file. Các bit của byte này nếu bằng 1 sẽ có ý nghĩa như sau:
		Bit 0: file chỉ được đọc; Bit 1: file ẩn; Bit 2: file hệ thống; Bit 3: Volume label; Bit 4: thư mục con Bit 5: archive; Bit 6: thiết bị nhớ khác (dùng cho hệ điều hành); Bit 7: không sử dụng
		Byte thuộc tính bằng 0x0F là dấu hiệu của file tên dài.
12	1	Dự phòng
13	1	Thời gian tạo file tính theo đơn vị 10ms, giá trị từ 0 đến 199
14	2	Thời gian tạo file theo format sau: bit 15-11: giờ (0-23); bit 10-5: phút (0-59); bit 4-0: giây/2 (0-29)
16	2	Ngày tạo file theo format sau. <i>Bit 15-9: năm (0-1980, 127 = 2107); bit 8-5: tháng (1-12); bit 4-0: ngày (1-31)</i>
18	2	Ngày truy cập cuối, theo format như ngày tạo file
20	2	2 byte cao của số thứ tự cluster đầu tiên của file trong FAT32
22	2	Thời gian sửa file lần cuối, theo format thời gian tạo file
24	2	Ngày sửa file lần cuối, theo format như ngày tạo file
26	2	Số thứ tự cluster đầu tiên của file trong FAT12/16.
28	4	Kích thước file tính bằng byte. Bằng 0 với thư mục con

- `int absread(int drive, int nsects, long lsect, void *buffer)`
 - `drive`: ổ đĩa cần đọc, A: 0, B:1, C:2
 - `nsects`: số sector cần đọc
 - `lsect`: vị trí sector bắt đầu đọc
 - `buffer`: vùng nhớ lưu nội dung thông tin cần đọc

Sinh viên viết chương trình trên C/C++ để thực hiện các nội dung sau:

.Đọc và in thông tin từ BOOT.

.Đọc, phân tích, hiển thị nội dung bảng FAT.

.Đọc, phân tích, hiển thị ROOT.

.Duyệt số thứ tự hoặc nội dung các **cluster** của file cho trước.

1. Viết chương trình để hiển thị thông tin boot sector
2. Viết chương trình đọc FAT và in nội dung 100 ô fat đầu tiên lên màn hình
3. Viết chương trình đọc ROOT và in nội dung giống lệnh DIR
4. Cho 1 tên file thuộc ROOT. Viết chương trình tìm tất cả các cluster của file đó
5. Viết chương trình đếm số cluster trống trong 100 cluster đầu tiên của ổ đĩa

Cho 1 tên file thuộc ROOT. Viết chương trình tìm tất cả các cluster của file đó

.Đọc Boot sector: lưu vào bs

.Đọc ROOT: lưu trong ROOT

.Đọc bảng FAT: lưu trong mảng FAT

.Giả sử tên file đưa vào lưu trong chuỗi tenfile;

.Kiểm tra xem file “tenfile” có nằm trong thư mục gốc???

1. - duyệt lần lượt các khoản mục (i từ 0 tới bs->root_entries) xem có khoản mục nào mà ROOT[i]->filename trùng với “tenfile” nhập vào hay không??
2. Giả sử ROOT[i]->filename == “tenfile” => khoản mục i trong thư mục gốc quản lý “tenfile” đang xét

Viết chương trình đếm số cluster trống của ổ đĩa

```
đọc boot sector  absread (2, 1, 0, bs);
```

```
Đọc FAT          absread(2, bs->sector_per_FAT, bs->reserved_sector,  
FAT);
```

```
Int dem=0;
```

```
For (int I =2; i<(bs->sector_per_FAT * bs->bytes_per_sector)/2; i++)
```

```
If (FAT[i]==0) dem++;
```

Hệ thống FAT16

Bài 1: Viết đoạn chương trình đếm số cluster trống trong 100 cluster đầu tiên của ổ đĩa D.

Bài 2: Viết đoạn chương trình in nội dung của 50 ô FAT đầu tiên của ổ đĩa C ra màn hình

Bài 3: Giả sử bảng FAT đã được đọc vào bộ nhớ tại địa chỉ `<< int *fat >>`. Giả sử một file được lưu trữ trên cluster đầu tiên là n. Viết đoạn chương trình liệt kê các cluster thuộc về file đó.